

PostgreSQL for the Absolute Beginner

Hour 1 of PostgreSQL Training Day

SCaLE LA 2026

About This Training

- Full day PostgreSQL training
- 6 hours with 90 minute lunch break
- Sample code and exercises available
- 3 experts to answer your questions

Today's Agenda

Hour	Topic
1	PostgreSQL for the Absolute Beginner
2	Getting Started with SQL in Postgres
3	Postgres DBA Basics
4	Postgres Troubleshooting
5	Configuration and Performance Tuning
6	Query Tuning

Training Materials

All slides, exercises, and Docker setup:

github.com/elizabeth-christensen/postgres-full-day-training

Sample database:

github.com/ryanbooz/bluebox



Let's Get Connected!

Setup Overview

1. Start PostgreSQL (Docker)
2. Install a psql client
3. Connect and load sample data
4. Verify everything works

~10-15 minutes

Prerequisites

You need [Docker Desktop](#) installed and running

- [Mac/Windows](#): docker.com/products/docker-desktop
- [Linux](#): Docker Engine + Docker Compose

Verify it's working:

```
docker --version
```

```
docker compose version
```

Step 1: Clone the Repository

Clone the training repo

```
git clone https://github.com/Snowflake-Labs/postgres-full-day-training.git
```

Navigate into the folder

```
cd postgres-full-day-training
```

Or download as ZIP from GitHub if you don't have git.

Step 2: Start PostgreSQL

Start PostgreSQL container

```
docker compose up -d
```

Verify it's running

```
docker ps
```

You should see postgres-training running on port 5432.

CONTAINER ID	IMAGE	STATUS	PORTS
abc123...	postgis/postgis:18-3.6	Up 10 seconds	0.0.0.0:5432->5432/tcp

Step 3: Install a psql Client

You need a way to connect to PostgreSQL. Choose one:

Mac

- brew install libpq (client only)
- brew install postgresql@18
- Postgres.app (includes psql)

Windows

- PostgreSQL installer ([postgresql.org](https://www.postgresql.org))
- pgAdmin (standalone)

Linux

- apt install postgresql-client
- yum install postgresql

Cross-Platform GUIs

- pgAdmin
- DBeaver
- TablePlus
- Azure Data Studio
- Beekeeper Studio

Mac: Homebrew (Recommended)

Install just the client tools (no server)

```
brew install libpq
```

Add to your PATH

```
echo 'export PATH="/opt/homebrew/opt/libpq/bin:$PATH"' >> ~/.zshrc
```

```
source ~/.zshrc
```

Verify

```
psql --version
```

Alternative: Install Postgres.app and use its bundled psql.

Windows / GUI Users

[pgAdmin](#) is a great option:

1. Download from [pgadmin.org](#)
2. Add a new server connection:
 - Host: localhost
 - Port: 5432
 - Username: postgres
 - Password: training

pgAdmin includes a Query Tool that works like psql.

Step 4: Connect to PostgreSQL

Connect using psql

```
psql postgresql://postgres:training@localhost:5432/postgres
```

You should see:

```
psql (18.x)
```

```
Type "help" for help.
```

```
postgres=#
```

You're connected! 🎉

Step 5: Create Database and Load Bluebox

From psql, create the database:

```
CREATE DATABASE bluebox;  
\c bluebox  
CREATE EXTENSION postgis;
```

Then exit psql (\q) and load the data:

Download Bluebox files

```
curl -LO https://raw.githubusercontent.com/ryanbooz/bluebox/main/bluebox_schema.sql  
curl -LO https://raw.githubusercontent.com/ryanbooz/bluebox/main/bluebox_data.sql.gz  
gunzip bluebox_data.sql.gz
```

Load schema and data

```
psql postgresql://postgres:training@localhost:5432/bluebox -f bluebox_schema.sql  
psql postgresql://postgres:training@localhost:5432/bluebox -f bluebox_data.sql
```

Step 6: Verify Your Setup

Connect to the bluebox database:

```
psql postgresql://postgres:training@localhost:5432/bluebox
```

Run a test query:

```
SELECT COUNT(*) FROM bluebox.film;
```

```
count
```

```
-----
```

```
7836
```

If you see 7836 films, you're all set! 

Troubleshooting: Last Resort

If you can't install psql locally, use Docker:

Connect via docker exec

```
docker exec -it postgres-training psql -U postgres -d bluebox
```

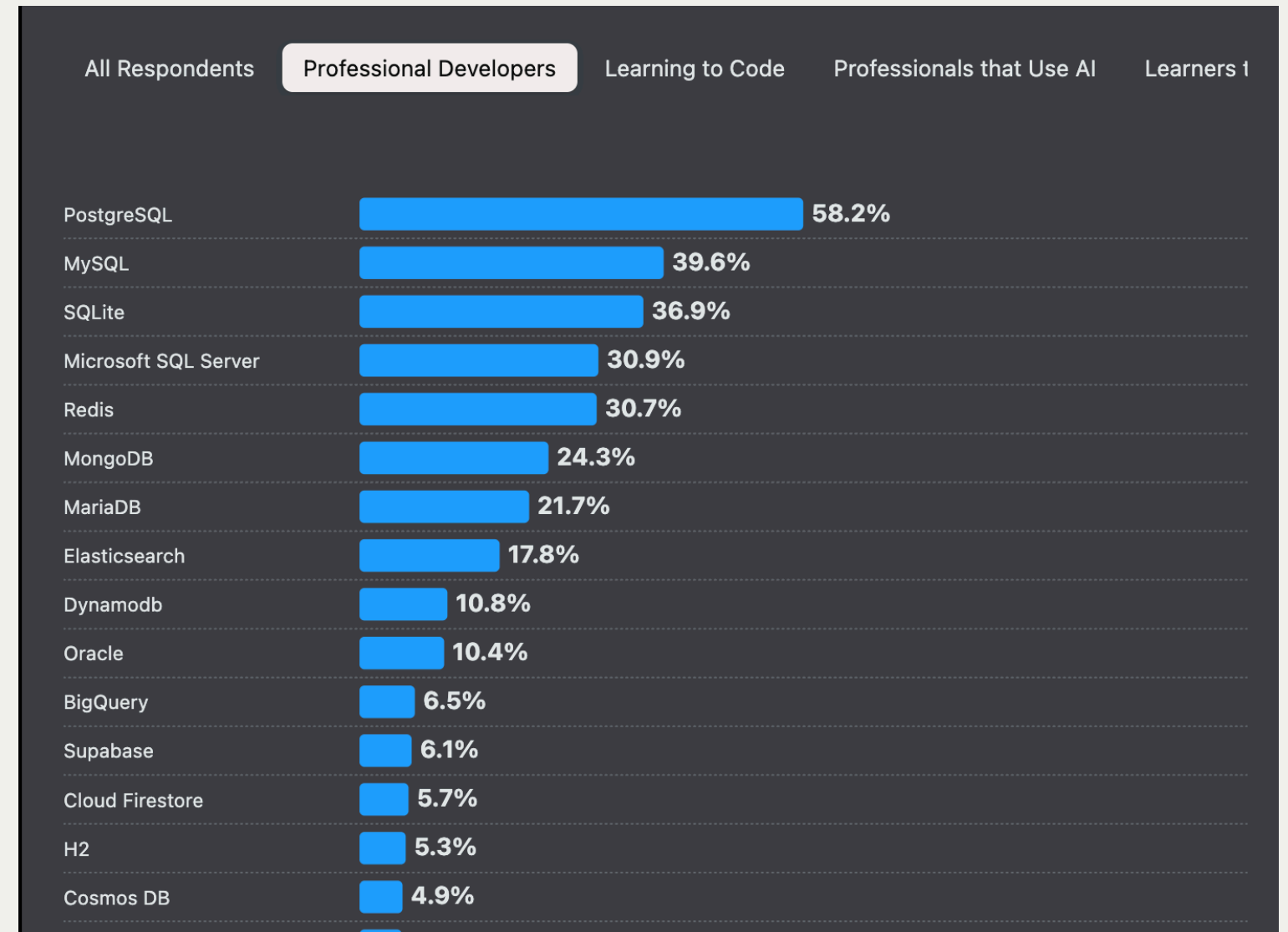
This works but isn't ideal for learning psql workflows.

Common issues:

- Docker not running → Start Docker Desktop
- Port conflict → Check if another Postgres is using 5432
- Permission denied → Run Docker Desktop as admin (Windows)

Why Postgres?

- **Open Source** - No licensing costs, ever
- **Standards compliant** - SQL standard with powerful extensions
- **Mature** - 35+ years of active development
- **Extensible** - PostGIS, pgvector, time series, and more
- **Reliable** - ACID compliant, battle-tested
- **Career** - Most loved database by developers



Running Postgres

Where can you run Postgres?

Running Postgres - Options

Local

- [Docker](#) (what we're using today)
- [Postgres.app](#) (macOS)
- [Homebrew](#) - brew install postgresql@18
- [Community packages](#) - postgresql.org/download

Cloud / Managed

- [AWS RDS](#) / Aurora
- [Azure Database](#)
- [Google Cloud SQL](#)
- [Crunchy Bridge](#) / Neon / Supabase

Connecting to PostgreSQL

Basic connection

```
psql -h localhost -p 5432 -U postgres -d bluebox
```

Connection string format (what we've been using)

```
psql postgresql://postgres:training@localhost:5432/bluebox
```

Connection to remote locations

protocol	user	Password	Host name	Port	Databas
postgres	postgres	JDcPwuav5DLpKoC6EsJ	@ p.vbjrfujv5beutaoelw72b.postgresbridge.com	5432	/meteors

GUI Tools

pgAdmin

- Official GUI tool
- Web-based interface
- Free and open source

DBeaver

- Multi-database support
- ER diagrams
- Free community edition

psql Introduction

psql - The Postgres CLI

The most powerful way to interact with Postgres

psql (18.x)

Type "help" for help.

bluebox=#

Essential psql Commands

Command	Description
<code>\l</code>	List all databases
<code>\c dbname</code>	Connect to database
<code>\dt</code>	List tables
<code>\d tablename</code>	Describe table
<code>\dn</code>	List schemas

- `\d` stands for "describe"

More psql Commands

Command	Description
<code>\du</code>	List users/roles
<code>\di</code>	List indexes
<code>\df</code>	List functions
<code>\x</code>	Toggle expanded display
<code>\timing</code>	Toggle query timing

psql Tips

-- Edit in your favorite editor

\e

-- Run SQL from a file

\i /path/to/script.sql

-- Output to file

\o output.txt

SELECT * FROM film;

\o

-- Get help on SQL commands

\h CREATE TABLE

psql Formatting

Default output can be messy when columns are wide:

```
id | name | email | long_column_that_wraps...
----+-----+-----+-----
1 | Alice | alice@test.com | this text wraps and becomes
| | | hard to read
```

Solution: \x auto - automatically switches to record format!

Expanded Display: \x auto

`\x auto` -- Let psql decide (recommended!)

`\x on` -- Always record

`\x off` -- Always horizontal (default)

Expanded view shows one column per line:

-[RECORD 1]-----

id | 1

name | Alice

email | alice@test.com

Pro tip: Put `\x auto` in your `~/.psqlrc` file!

Pretty Unicode Borders

```
\pset linestyle unicode
\pset border 2
```

id	name	age
1	Alice	30
2	Bob	25

Makes output much easier to read!

More Useful psql Settings

```
\timing          -- Show query execution time
\conninfo        -- Show current connection info
\pset pager off  -- Turn off page-by-page scrolling
```

```
bluebox=# \timing
```

Timing is on.

```
bluebox=# SELECT COUNT(*) FROM film;
```

```
count
```

```
-----
```

```
7836
```

```
(1 row)
```

```
Time: 2.145 ms
```

```
bluebox=# \conninfo
```

You are connected to database "bluebox" as user "postgres"
on host "localhost" at port "5432".

Making NULLs Visible

By default, NULL displays as blank - easy to confuse with empty string!

```
\pset null '🍀'  
SELECT title, budget FROM bluebox.film WHERE budget IS NULL LIMIT 3;
```

title	budget
-----+	
Dune: Part Two	🍀
Migration	🍀
Mean Girls	🍀

Now you can clearly see which values are actually NULL!

Users and Permissions

Postgres Roles

In Postgres, **users** and **groups** are both **roles**

-- Create a user (role with login)

```
CREATE ROLE app_user WITH LOGIN PASSWORD 'secure_password';
```

-- Create a group (role without login)

```
CREATE ROLE readonly;
```

Role Attributes

```
CREATE ROLE app_user WITH  
LOGIN  
PASSWORD 'secure_password'  
CREATEDB  
CREATEROLE;
```

Common attributes:

- LOGIN / NOLOGIN
- SUPERUSER / NOSUPERUSER
- CREATEDB / NOCREATEDB
- CREATEROLE / NOCREATEROLE

⚠ Password Security: Don't Do This!

-- This works, BUT the password may be logged in plaintext!

```
CREATE ROLE app_user WITH PASSWORD 'secret123!' LOGIN;
```

If you use pg_audit or any query logging, the plaintext password is now in your logs!
😱

Instead, use \password:

```
\password app_user
```

```
Enter new password: *****
```

```
Enter it again: *****
```

\password hashes the password **before** sending to the server

Password Hashing in Postgres

Postgres uses **SCRAM-SHA-256** (since v10):

- **Hashing $\neq$ Encryption** - Hashing is one-way, cannot be reversed
- **Salt** - Random data added before hashing, so identical passwords produce different hashes
- **Challenge-Response** - Password never sent in plaintext over the network

Hashed passwords are stored in `pg_authid.rolpassword`

Password Best Practices

✓ Use `\password` to create/change passwords (pre-hashes)

✓ Use `SCRAM-SHA-256` (default since Postgres 10)

✓ Migrate old MD5 passwords - MD5 is deprecated in Postgres 18!

✓ Enforce password policies - length, complexity, rotation

-- Check if you have old MD5 passwords

```
SELECT rolname, rolpassword
FROM pg_authid
WHERE rolpassword LIKE 'md5%';
```

Granting Privileges

-- Grant connection to database

```
GRANT CONNECT ON DATABASE bluebox TO app_user;
```

-- Grant schema usage

```
GRANT USAGE ON SCHEMA bluebox TO app_user;
```

-- Grant table privileges

```
GRANT SELECT ON ALL TABLES IN SCHEMA bluebox TO app_user;
```

-- Grant specific privileges

```
GRANT SELECT, INSERT, UPDATE ON bluebox.rental TO app_user;
```

Manage Privileges with Groups

Roles can be members of other roles (like groups!)

-- Create a group role for data analytics team

```
CREATE ROLE data_analytics NOLOGIN;
```

-- Grant read access on Bluebox to the group

```
GRANT CONNECT ON DATABASE bluebox TO data_analytics;
```

```
GRANT USAGE ON SCHEMA bluebox TO data_analytics;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA bluebox TO data_analytics;
```

Grant Users Role Membership

-- Create a user for an analyst

```
CREATE ROLE maria LOGIN;  
\password maria
```

-- Add maria to the data_analytics group

```
GRANT data_analytics TO maria;
```

-- Maria now inherits all permissions from data_analytics!

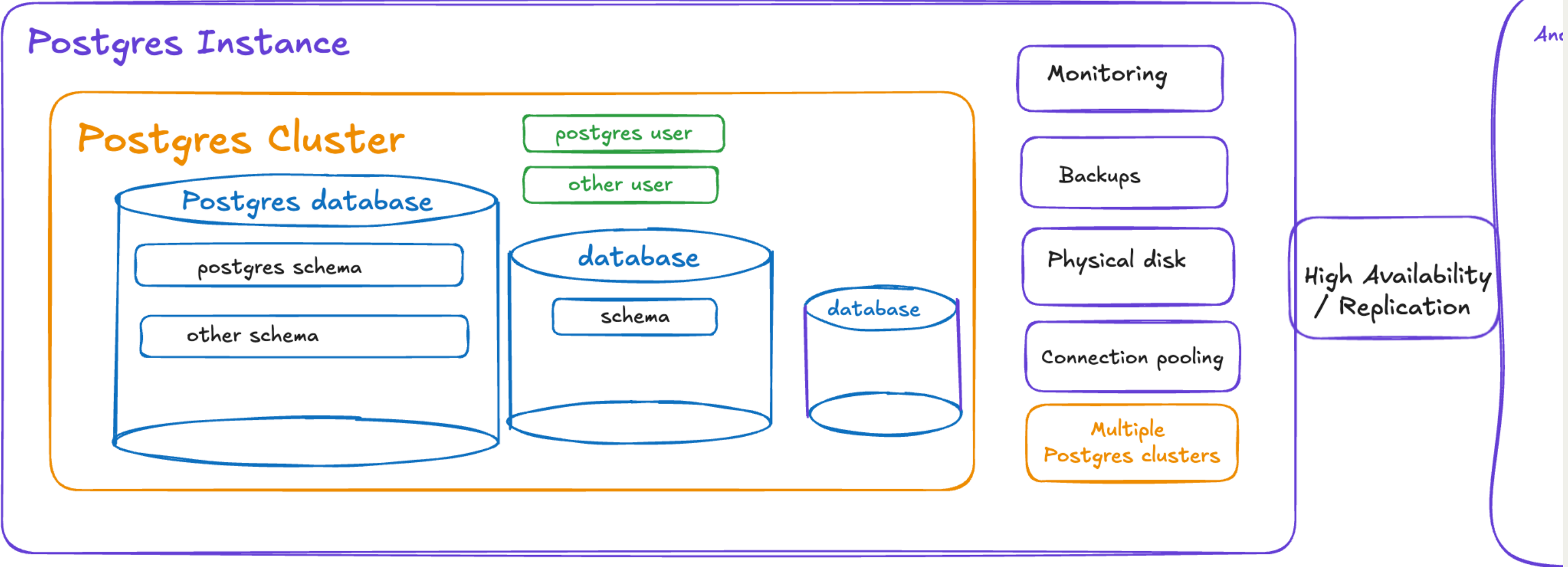
View Role Memberships

\du

List of roles		
Role name	Attributes	Member of
-----+-----+-----		
data_analytics	Cannot login	{}
maria	{data_analytics}	
app_user	{}	
postgres	Superuser, Create role, ...	{}

Maria is a member of data_analytics - she inherits its permissions!

Schemas



What is a Schema?

A **schema** is a namespace within a database

- Organizes database objects (tables, views, functions)
- Provides access control boundaries
- Avoids naming conflicts

Default Schema

-- The default schema is 'public'

```
CREATE TABLE my_table (id int);
```

-- Same as:

```
CREATE TABLE public.my_table (id int);
```

Creating Schemas

-- Bluebox already has its schema, but you could create more:

`CREATE SCHEMA` reporting;

-- Then create tables in that schema

`CREATE TABLE` reporting.daily_stats (...);

Bluebox uses bluebox schema to organize all its tables

Schema Search Path

-- View current search path

```
SHOW search_path;
```

-- Output: "\$user", public

-- Set search path

```
SET search_path TO bluebox, public;
```

-- Now queries will look in bluebox first

```
SELECT * FROM film; -- Same as bluebox.film
```

Bluebox Schema

The Bluebox database uses a bluebox schema with 17 tables:

```
SET search_path TO bluebox, public;  
\dt
```

Schema		tablename	
-----+-----			
bluebox		customer	(186,740 rows)
bluebox		film	(7,836 rows)
bluebox		person	(258,772 rows)
bluebox		store	(196 rows)
...and 13 more tables			

ALTER User Search Path

-- Set search path

```
ALTER USER maria SET search_path TO bluebox, public;
```

-- From next login, queries will look in bluebox first

```
SELECT * FROM film; -- Same as bluebox.film
```

Object and Data Types

Database Objects

- **Tables** - Store data in rows and columns
- **Views** - Saved queries that act like tables
- **Indexes** - Speed up data retrieval
- **Sequences** - Auto-incrementing number generators
- **Functions** - Reusable SQL/procedural code

Common Data Types

Numeric

- INTEGER / BIGINT
- NUMERIC(p,s)
- REAL / DOUBLE

Character

- TEXT (preferred)
- VARCHAR(n)

Date/Time

- DATE / TIME
- TIMESTAMP
- TIMESTAMPTZ ★
- INTERVAL

Other

- BOOLEAN
- UUID
- JSON / JSONB
- array of any other type, e.g. int[], text[]



Time: Use TIMESTAMPTZ!

TIMESTAMP vs TIMESTAMPTZ - always prefer **TIMESTAMPTZ**

-- **TIMESTAMP**: No timezone info (dangerous!)

CREATE TABLE events (event_time **TIMESTAMP**);

-- **TIMESTAMPTZ**: Stores in UTC, converts on display ✓

CREATE TABLE events (event_time **TIMESTAMPTZ**);

-- What time is it?

SELECT NOW(); -- 2026-01-16 10:30:00-08

TIMESTAMPTZ handles daylight saving automatically!



Use NUMERIC, Not MONEY!

-- DON'T use the MONEY type

price MONEY -- ✗ Locale-dependent, rounding issues

-- DO use NUMERIC for currency

price NUMERIC(10,2) -- ✓ Exact precision, no surprises

Bluebox uses NUMERIC for payment amounts:

```
SELECT amount FROM bluebox.payment LIMIT 3;
```

```
amount
```

```
-----
```

```
1.99
```

```
1.99
```

```
3.98
```

NUMERIC stores exact values - no floating point errors!



Custom Data Types: ENUM

Postgres lets you create custom types!

Bluebox uses an ENUM for MPAA ratings:

```
CREATE TYPE mpaa_rating AS ENUM (  
    'G', 'PG', 'PG-13', 'R', 'NC-17', 'NR'  
);
```

-- Used in the film table

```
SELECT title, rating FROM bluebox.film  
WHERE rating = 'PG-13' LIMIT 3;
```

title	rating
Are You There God? It's Me, Margaret.	PG-13
Love at First Sight	PG-13
Batman Returns	PG-13

Creating a Table

Let's add a customer reviews feature to Bluebox!

```
CREATE TABLE bluebox.customer_review (  
  review_id SERIAL PRIMARY KEY,  
  customer_id INT REFERENCES bluebox.customer(customer_id),  
  film_id BIGINT REFERENCES bluebox.film(film_id),  
  rating SMALLINT CHECK (rating BETWEEN 1 AND 5),  
  review_text TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Add Yourself as a Customer

-- Add yourself to the Bluebox database

```
INSERT INTO bluebox.customer
(customer_id, store_id, full_name, email)
VALUES (
  (SELECT MAX(customer_id) + 1 FROM bluebox.customer),
  1, -- Store #1
  'Your Name Here', -- Your name!
  'you@example.com'
)
RETURNING customer_id, full_name;
```

```
customer_id | full_name
```

```
-----+-----
```

```
205026 | Your Name Here ← Save this ID!
```

Note: this is a great use of the Postgres feature RETURNING

Find Films to Review

-- Find some popular films to review

```
SELECT film_id, title FROM bluebox.film
WHERE title IN ('The Dark Knight', 'Inception',
               'Interstellar', 'Dune: Part Two');
```

film_id	title
155	The Dark Knight
27205	Inception
157336	Interstellar
693134	Dune: Part Two

Insert Your Reviews

Replace 205026 with your actual customer_id!

-- Use YOUR customer_id from the previous step!

INSERT INTO bluebox.customer_review

(customer_id, film_id, rating, review_text)

VALUES

(205026, 155, 5, 'Heath Ledger was incredible!'),

(205026, 27205, 5, 'Mind-bending! Had to watch twice'),

(205026, 693134, 5, 'Even better than the first Dune!');

Querying Your Reviews

-- See your reviews with a JOIN

```
SELECT c.full_name, f.title, r.rating,  
       LEFT(r.review_text, 30) as review  
FROM bluebox.customer_review r  
JOIN bluebox.customer c ON r.customer_id = c.customer_id  
JOIN bluebox.film f ON r.film_id = f.film_id  
ORDER BY r.created_at DESC;
```

full_name		title		rating		review
-----+-----+-----+-----						
Your Name		The Dark Knight		5		Heath Ledger was incredible
Your Name		Inception		5		Mind-bending! Had to watch
Your Name		Dune: Part Two		5		Even better than the first

Bluebox Tables

\d bluebox.film

Column	Type	Description
film_id	bigint	Primary key
title	text	Movie title
overview	text	Plot summary
release_date	date	Release date
vote_average	real	TMDB rating (0-10)
popularity	real	TMDB popularity score
budget	bigint	Production budget
revenue	bigint	Box office revenue

Extensions

What are Extensions?

Extensions add functionality to Postgres:

- New data types
- New functions
- New operators
- New index types

Contrib Extensions

Bundled with Postgres - just CREATE EXTENSION name;

Extension	What it does
pg_stat_statements	Track query performance statistics
pg_trgm	Fuzzy text search with trigrams
citext	Case-insensitive text type
hstore	Key-value store in a column
btree_gist	GiST index support for common types
uuid-osp	Generate UUIDs
tablefunc	Crosstab / pivot table queries
pgcrypto	Encryption functions

Popular Third-Party Extensions

- [PostGIS](#) - Geospatial data and queries
- [pgvector](#) - Vector similarity search (AI/ML)
- [TimescaleDB](#) - Time-series data
- [pg_cron](#) - Job scheduling
- [HypoPG](#) - Hypothetical indexes

Hour 1 Summary

-  Setup: Docker, psql client, Bluebox loaded
-  Why Postgres is a great choice
-  Connecting with psql and GUI tools
-  Essential psql commands
-  Users, roles, and permissions
-  Schemas for organization
-  Data types and table creation
-  Extensions for added functionality

Questions?

Next: Getting Started with SQL in Postgres